

Maintenance Planner Project Report

[!NOTE] This report provides a comprehensive overview of the **Usage-Based Preventive Maintenance Planner for Shared High-Value Diagnostic Equipment** project.

1. Executive Summary

The project is an end-to-end working prototype for a diagnostic centre managing shared high-value medical equipment (e.g., MRI, CT, X-Ray). It transitions the centre from a static, calendar-based maintenance scheduling approach to a **usage-based** system. The primary goal is to reduce unplanned downtime and optimize operational continuity by prioritizing equipment maintenance based on real-world utilisation, age, fault history, and device criticality.

[!WARNING] **Medical Disclaimer:** The system provides operational maintenance planning support only. It does not provide medical advice, diagnosis, treatment recommendations, or equipment safety certification.

2. Architecture & Technology Stack

The application is built using a modern, data-driven Python stack:

- **Frontend & UI:** [Streamlit](#) for building an interactive, role-based web dashboard.
- **Backend Logic:** Python (Pandas, NumPy) for data processing and risk calculations.
- **Database:** SQLite (database/maintenance.db) for persistent storage of equipment state, synthetic data, and audit logs.
- **Visualization:** Plotly for interactive data visualizations (e.g., risk distribution, utilisation charts).
- **Testing:** pytest for unit testing the risk scoring and data processing modules.

3. Core Modules and Implementation

3.1 Synthetic Data Generation

File: [generate_data.py](#)

To demonstrate the system's capabilities without exposing real PHI or hospital data, the project includes a robust synthetic data generator. It produces realistic data for: - **Equipment:** Details like age, utilisation, criticality, and service intervals. - **Service History:** Past maintenance records and issues found. - **Fault Codes:** Simulated hardware and software warnings with varying severities. - **Bookings:** Operational schedules to simulate hard constraints during planning. - **Technicians:** Staff availability and specializations.

It also deliberately injects edge cases (e.g., bad data, negative hours, missing service history) to test the robustness of the data cleaning pipeline.

3.2 Feature Engineering & Risk Engine

Files: [src/feature_engineering.py](#), [src/risk_engine.py](#)

The system evaluates equipment health by calculating normalized scores (0-100) across five dimensions: 1. **Utilisation Score:** Based on actual operating hours versus available hours. 2. **Age Score:** Based on the equipment's age in years (capping at 15 years as maximum risk). 3. **Criticality Score:** Based on operational importance (Low, Medium, High, Critical). 4. **Fault Score:** Based on the frequency and severity of recent error codes. 5. **Service Score:** Based on the time elapsed since the last service against the recommended maintenance interval.

The Risk Engine computes an **Overall Risk Score** using an Admin-configurable weighted sum formula (default weights): $\text{python Risk} = (0.30 * \text{Utilisation}) + (0.20 * \text{Age}) + (0.20 * \text{Fault}) + (0.15 * \text{Service}) + (0.15 * \text{Criticality})$

3.3 Maintenance Planner

File: [src/planner.py](#)

The planning algorithm generates maintenance schedules by balancing the risk scores with real-world constraints: - **Hard Constraints:** Ensures Technician availability, avoids conflicts with existing patient bookings, and fits within the operational window. - **Soft Constraints:** Attempts to schedule during historically low-utilisation windows to minimize disruption.

3.4 Audit & Accountability

File: [src/audit.py](#)

Every critical action taken in the system is securely logged into the SQLite database (audit_logs table). This includes logins, consent acknowledgments, configuration changes, and schedule overrides. This ensures full traceability and accountability for compliance.

4. Workflows and Role-Based Access Control

The application implements a strict Role-Based Access Control (RBAC) model to ensure the right personnel have the correct permissions.

User Roles:

- **Admin (admin):** Can configure risk weights, manage users, and has full system access.

- **Maintenance Manager (manager):** Responsible for reviewing recommendations, approving schedules, or explicitly overriding them (which requires a logged justification).
- **Technician (technician):** Logs in to view assigned tasks and update maintenance statuses.
- **Operations Staff (operations):** Views equipment availability and scheduling to coordinate patient care without disruption.

Expected Workflow:

1. **Data Ingestion:** The system reads the latest state of equipment, faults, and bookings.
2. **Analysis:** The Risk Engine calculates risk scores and breakdowns for all devices.
3. **Recommendation:** The Planner finds feasible maintenance slots.
4. **Review & Approval:** The Maintenance Manager reviews the proposed slots on the Dashboard. They can either **Approve** or **Override** (providing a reason).
5. **Execution:** The Technician performs the scheduled maintenance.

5. Evaluation and Metrics

The system includes an **Evaluation Module** that continuously measures the reduction in unplanned downtime. It compares the proposed **Usage-Based strategy** against a **Calendar-Only baseline**. The primary objective is to demonstrate at least a 20% reduction in unplanned downtime, proving the operational value of transitioning to a dynamic, risk-adjusted maintenance schedule.

6. Limitations and Future Improvements

[!TIP] **Future Roadmap**

- **Real-world Data Integration:** The current synthetic data may not perfectly mirror real hospital behavior. Future phases should focus on integrating live IoT telemetry and actual hospital booking APIs (e.g., HL7/FHIR).
- **Weight Calibration:** The risk calculation weights are currently operational assumptions and will require real-world tuning and validation.
- **Predictive ML Models:** The current risk scoring is heuristic and rule-based. Future iterations can incorporate Time-Series Forecasting and Machine Learning models (like XGBoost or LSTMs) to predict failure events more accurately.
- **Regulatory Compliance:** The tool currently serves as an operational aid. It does not replace regulatory maintenance schedules mandated by manufacturers or health authorities.